Store or Recompute? Characterizing the Carbon Implications of KV Cache Retention in LLM Inference

Abnash Bassi
Carnegie Mellon University

Jaylen Wang
Carnegie Mellon University

Fiodar Kazhamiaka
Microsoft

Daniel S. Berger
Microsoft

Akshitha Sriraman
Carnegie Mellon University

Abstract

The growing compute demand of Large Language Model (LLM) inference carries a significant environmental cost. To reduce GPU load, intermediate computation (KV caches) can be saved in non-volatile storage servers and retrieved when the session becomes active. This setup presents a fundamental tradeoff between high power and associated operational carbon emissions of recomputing the KV cache versus the high capacity requirements and embodied carbon emissions of storage. We present a first-order analytical model to identify break-even points between the two choices as a function of storage duration, comparing the embodied carbon of storage hardware against the operational carbon of GPU-based recomputation. We further examine how response time requirements interact with carbon-saving decisions, revealing scenarios where environmentally preferable strategies may conflict with performance needs. Our results demonstrate that the break-even duration varies significantly with different system parameters (e.g., the carbon intensity of the energy source), suggesting opportunities for sustainability-aware retention policies.

1 Introduction

Large Language Model (LLM) inference is deploying at scale, driving a massive demand for computation [8]. This growing compute demand incurs a significant environmental cost [1, 9, 27]. To reduce redundant computation across queries, inference systems generate and store intermediate key-value (KV) caches [7, 18]. Upon processing a follow-up query, e.g., in a multi-turn conversation, the system can reuse this KV-cache rather than recomputing it from scratch. However, due to the KV cache's large memory footprint [4, 7] and the potentially long duration between reuse, inference systems face a retention decision: store the KV cache on persistent storage (e.g., an SSD) for future reuse, or discard it and recompute it when the next query arrives.

Between these two choices, we identify a critical environmental tradeoff. Storing KV caches on disk for long durations increases *embodied emissions* because provisioning sufficient storage capacity requires manufacturing additional hardware. Recomputing KV caches avoids these embodied emissions but increases *operational emissions* due to the GPU's high power draw during the recomputation step. The carbon-saving choice depends on the *reuse distance*—the time between when a KV cache is first computed and when a follow-up query requires it. Prior work [2, 29] has examined how to manage KV cache storage capacity and hierarchy for performance, but how this retention decision's environmental impact changes with reuse distance, token length, and other system parameters remains uncharacterized.

To address this gap, we perform the first holistic characterization of the carbon tradeoff between KV cache storage and recomputation. We present a first-order analytical model that identifies the break-even points between the two strategies as a function of realistic system parameters: KV cache size, reuse distance, GPU device, storage technology, and grid carbon intensity. The model compares the embodied carbon of provisioning storage hardware with the operational carbon of GPU-based recomputation.

Beyond carbon analysis, we examine how performance requirements interact with carbon-saving decisions. Specifically, we analyze the tension between minimizing response time and minimizing emissions. Our analysis reveals scenarios where environmentally preferable strategies conflict with strict latency Service Level Objectives (SLOs). For example, we find that retrieving a cache from storage to save operational carbon may violate a real-time application's latency constraint. Thus, we reveal scenarios where looser latency constraints, e.g., a user is willing to wait longer for a response, can be exploited for lower added carbon emissions, ranging from 2-5× for the durations studied. Our results show that the break-even duration, and thus the storage-versus-recompute tradeoff, varies significantly based on cache size. We find that for low carbon intensities (e.g., during the day when more renewables are available), storing the cache becomes carbon-saving after $\sim 3-4$ minutes, whereas for higher carbon intensities, recomputation remains superior for ~ 7 minutes. Additionally, depending on the LLM and hardware configuration used, the saving duration determined from our model can be on the order of hours.

These findings suggest that static policies are insufficient, creating an opportunity for sustainability-aware retention

HotInfra '26, Raleigh, NC, USA

.

Table 1. Summary of notation.

Symbol	Description
S_{KV}	KV cache size (GB)
S_{SSD}	SSD capacity (GB)
t_{store}	Duration the KV cache is stored on the SSD
t_{recomp}	Time to recompute the KV cache on the GPU
t_{life}	Hardware operational lifetime
P_{SSD}	SSD idle power draw (W)
P_{GPU}	GPU active power draw (W)
P_{other}	Non-GPU component power draw (W)
CI	Grid carbon intensity (gCO_2eq/kWh)
PUE	Power Usage Effectiveness
B	Batch size (concurrent requests)
c_{SSD}	Embodied carbon per GB of SSD (gCO_2eq/GB)
c_{GPU}	Embodied carbon of the GPU system (gCO_2eq)

policies that dynamically select between storage and recomputation. In particular, such a system would need to predict "reuse distance", i.e., the likelihood of a follow-up request and the duration until the request, and make a decision accordingly of whether to store or to recompute a KV cache.

2 Methodology

We model the *added* carbon footprint of two KV cache management strategies: (1) storing the cache on an SSD for later retrieval (§2.1), and (2) discarding the cache and recomputing it when a follow-up query arrives (§2.2). We also estimate the latency of each strategy to identify carbon-performance tradeoffs. Table 1 summarizes the notation used throughout this section.

2.1 Storage Scenario

The storage scenario retains the KV cache on an SSD for a duration t_{store}, representing the reuse distance.

Operational carbon. The SSD consumes power while holding the cache. We compute the energy drawn during t_{store}, scaled to the fraction of SSD capacity the KV cache occupies: $E_{\text{store}} = P_{\text{SSD}} \cdot t_{\text{store}} \cdot \frac{S_{\text{KV}}}{S_{\text{SSD}}}$. Multiplying by the grid carbon intensity yields the operational carbon: $\Delta C_{\text{op}}^{\text{store}} = CI \cdot E_{\text{store}}$.

Embodied carbon. As with prior work [3, 26], we amortize the SSD's embodied carbon over its operational lifetime. The per-GB embodied cost c_{SSD} (derived from prior work [23]) is scaled by the cache size and the fraction of the SSD's lifetime the cache occupies: $\Delta C_{\text{emb}}^{\text{store}} = c_{\text{SSD}} \cdot S_{\text{KV}} \cdot \frac{t_{\text{store}}}{t_{\text{life}}}$.

Total carbon. The added carbon footprint of storing the KV cache is: $\Delta C_{\text{total}}^{\text{store}} = \Delta C_{\text{op}}^{\text{store}} + \Delta C_{\text{emb}}^{\text{store}}$.

2.2 Recompute Scenario

The recompute scenario discards the KV cache after the initial query and regenerates it from scratch when the follow-up query arrives. The added carbon is attributed to the GPU cluster that performs the recomputation.

Operational carbon. During recomputation, the GPU operates near its maximum rated power [6] because KV cache generation (the prefill phase) is compute-bound. The total system power includes the GPU, non-GPU components (e.g., host CPU, DRAM, networking), and facility overhead captured by PUE [13]: $P_{\text{sys}} = (P_{\text{GPU}} + P_{\text{other}}) \cdot PUE$. The recompute time t_{recomp} depends on the number of input tokens, model architecture, and hardware. Based on internal profiling, we use a piecewise model: t_{recomp} scales linearly with token count for sequences ≤8k tokens and quadratically for sequences >8k tokens. The recompute energy is then: $E_{\text{recomp}} = P_{\text{sys}} \cdot t_{\text{recomp}}$. We assume a batch of B requests recompute their KV caches in parallel across the GPU cluster. Each request's operational carbon is: $\Delta C_{\text{op}}^{\text{recomp}} = \frac{CI \cdot E_{\text{recomp}}}{B}$.

Embodied carbon. We amortize the GPU system's total embodied carbon c_{GPU} (including GPU dies and HBM) over its operational lifetime, then attribute a share proportional to t_{recomp} and divide across the batch: $\Delta C_{\text{emb}}^{\text{recomp}} = c_{\text{GPU}} \cdot \frac{t_{\text{recomp}}}{t_{\text{life}}} \cdot \frac{1}{B}$.

Total carbon. The added carbon footprint of recomputing the KV cache is: $\Delta C_{\text{total}}^{\text{recomp}} = \Delta C_{\text{op}}^{\text{recomp}} + \Delta C_{\text{emb}}^{\text{recomp}}$.

2.3 Break-Even Reuse Distance

The storage scenario's carbon cost grows with t_{store} , while the recompute scenario's cost is independent of t_{store}. The *break-even reuse distance* t_{store}^* is the storage duration at which both strategies produce the same added carbon: $\Delta C_{\text{total}}^{\text{store}}(t_{\text{store}}^*) = \Delta C_{\text{total}}^{\text{recomp}}$. Substituting prior equations and solving for t_{store}^*: $t_{\text{store}}^* = \frac{\Delta C_{\text{op}}^{\text{recomp}} + \Delta C_{\text{emb}}^{\text{recomp}}}{CI \cdot P_{\text{SSD}} \cdot \frac{S_{\text{KV}}}{S_{\text{SSD}}} + c_{\text{SSD}} \cdot S_{\text{KV}} \cdot \frac{1}{t_{\text{life}}}}$. For reuse distances shorter than t_{store}^*, storing the KV cache produces fewer emissions than recomputing. For reuse distances longer than t_{store}^*, recomputing is carbon-saving. By varying parameters like CI, S_{KV}, and GPU device, we examine how the break-even point shifts across different deployment configurations in §4.

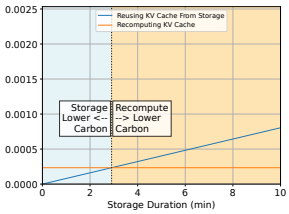
3 Implementation

We now detail the configuration parameters and assumptions used to evaluate the carbon-saving storage duration.

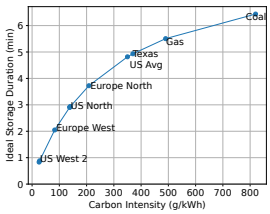
Hardware configuration. Similar to prior work [17], we model an Nvidia HGX H100 system containing 8 GPUs, each with 80GB of HBM. We model our SSDs after the Seagate Nytro 3331 SSD with 960GB capacity [21].

Embodied carbon data. We use ACT to calculate the embodied emissions of the SSD [3]. We derive the embodied emissions of the 8-GPU system from manufacturer reports [16]. We assume a six-year lifetime [12].

Operational carbon data. The electricity grid's carbon intensity affects the operational carbon emissions of the storage and recompute scenarios. We use carbon intensities reported for Azure data center regions and for specific energy types, such as coal and natural gas, from prior work [20, 26].



(a) Additional carbon emissions versus storage duration when reusing the KV cache from storage versus recomputing the KV cache. We save carbon by storing the KV cache until the break-even point and then recomputing past that point.



(b) Ideal storage duration for varying electricity grid carbon intensity. As grid CI increases, we save carbon by storing the KV cache for longer to avoid recomputing.

We use the Power Usage Effectiveness (*PUE*) reported by Microsoft Azure [13]. Component power consumption is determined from manufacturer reports [15, 21].

Model parameters. We use Meta's Llama-2-70b model for our evaluation [25]. We calculate the KV cache size as a function of the number of tokens and model parameters, similar to existing model size calculators [11]. For the compute time, we use a piecewise function mapping the number of input tokens to the time-to-first-token (TTFT). To calculate the TTFT, we assume a linear relationship between the number of tokens and TTFT for ≤8k tokens and use llama-2 data from prior work to construct a linear equation [17]. For the TTFT of a large number of input tokens (>8k tokens), we fit a quadratic function to prior internal data.

4 Results

Fig. 1a shows the added carbon for varying KV cache storage durations. The intersection identifies the break-even point (ideal storage duration) for the Llama-2-70b model based on 8000 input tokens, batch size of 512, and the average carbon intensity from the U.S. electricity grid. The shaded region to the left of the break-even points indicates storing the KV cache is ideal from a carbon standpoint when the duration of storage is less than ∼ 3 minutes. Beyond this duration, recomputing the KV cache using the GPU would lower the carbon footprint compared to keeping it in storage. This means that for a user in an ongoing session with LLM chat bot with 8000 tokens per query, if they step away from the conversation for durations longer than ∼ 3 minutes, it is advantageous to discard the KV cache and recompute later.

At higher carbon intensities, it is beneficial to store the KV cache in storage for longer rather than recompute. Fig. 1b shows how the carbon-saving duration of storage increases as the carbon intensity of the electricity grid powering the LLM inference hardware increases, with the same setup as Fig. 1a. This is because the carbon emissions of recomputing are primarily due to operational carbon emissions, which increase with the carbon intensity, resulting in higher overall carbon emissions for recomputing. Since the ideal duration to store the KV cache will be different depending on the region of operation, a scheduler would be well-suited to determine whether or not a KV cache should be stored; we leave this for future work.

We also present a first-order estimate of the latency to read the stored KV cache from SSD. The TTFT for storing the KV cache is 2.2195s whereas recomputing is 0.5067s. We assume the SSD is located on the same server and thus exclude network delay. Although the TTFT is higher for the storage scenario and may make it challenging to meet SLOs, this could be reduced in practice by techniques such as striping and KV cache compression.

5 Related Work

We present, to the best of our knowledge, the first *break-even* analysis that considers carbon and storage duration for KV cache storage versus recompute. Prior work that determines whether to store or recompute a KV cache has been either been limited to considering cost and power [22], does not consider both storage and recomputing scenarios [14], or does not evaluate the break-even point based on storage duration to determine whether a KV cache should be stored or recomputed [24]. Reducing the KV cache size based on compression techniques proposed in prior work is beneficial to reduce the embodied carbon associated with storage [19], [5], [10]. Some work also limits the KV cache size by implementing an eviction policy [28]. Prior work [10] has shown reduced network latency by compressing the KV cache. However, there can also be an increase in latency associated with time spent compressing and decompressing the KV cache. Although KV cache compression is a complementary approach to possibly reducing the carbon footprint and latency of KV cache storage, as the context lengths used for LLM inference continue to grow, there remains a need to evaluate when to store or recompute a KV cache.

6 Conclusion

This work characterizes the environmental tradeoff between storing LLM KV caches on persistent hardware and recomputing them on GPUs. The analytical model identifies specific break-even reuse distances where the embodied emissions of storage hardware equals the operational emissions of GPU-based recomputation. Results demonstrate that these break-even points shift significantly based on grid carbon intensity, model scale, and hardware specifications.

The analysis also reveals a fundamental tension between carbon-saving decisions and latency constraints. In several scenarios, the retrieval of a stored cache minimizes emissions but may violate strict performance Service Level Objectives (SLOs). Conversely, recomputation provides faster response times at a higher environmental cost.

References

[1] Andrew A Chien, Liuzixuan Lin, Hai Nguyen, Varsha Rao, Tristan Sharma, and Rajini Wijayawardana. 2023. Reducing the Carbon Impact of Generative AI Inference (Today and in 2035). In *Workshop on Sustainable Computer Systems*.

[2] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-Turn Conversations with CachedAttention. In *Annual Technical Conference*.

[3] Udit Gupta, Mariam Elgamal, Gage Hills, Gu-Yeon Wei, Hsien-Hsin S Lee, David Brooks, and Carole-Jean Wu. 2022. ACT: Designing Sustainable Computer Systems With an Architectural Carbon Modeling Tool. In *Annual International Symposium on Computer Architecture*.

[4] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun S Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *Advances in Neural Information Processing Systems* (2024).

[5] Bo Jiang, Taolue Yang, Youyuan Liu, Xubin He, Sheng Di, and Sian Jin. 2026. PackKV: Reducing KV Cache Memory Footprint through LLM-Aware Lossy Compression. arXiv:2512.24449

[6] Aditya K Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2025. Pod-attention: Unlocking full prefill-decode overlap for faster llm inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 897–912.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Symposium on Operating Systems Principles*.

[8] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. 2024. LLM Inference Serving: Survey of Recent Advances and Opportunities. In *High Performance Extreme Computing Conference*.

[9] Pengfei Li, Jianyi Yang, Mohammad A Islam, and Shaolei Ren. 2025. Making AI Less 'Thirsty'. *Commun. ACM* 68 (2025).

[10] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) *(ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 38–56.

[11] LMCache. 2025. KV Cache Size Calculator — lmcache.ai. https://lmcache.ai/kv_cache_calculator.html.

[12] Jialun Lyu, Jaylen Wang, Kali Frost, Chaojie Zhang, Celine Irvene, Esha Choukse, Rodrigo Fonseca, Ricardo Bianchini, Fiodar Kazhamiaka, and Daniel S. Berger. 2023. Myths and Misconceptions Around Reducing Carbon Embedded in Cloud Platforms. In *Proceedings of the 2nd Workshop on Sustainable Computer Systems* (Boston, MA, USA) *(HotCarbon '23)*. Association for Computing Machinery, New York, NY, USA, Article 7, 7 pages.

[13] Microsoft. 2024. Microsoft 2024 Environmental Sustainability Report. https://cdn-dynmedia-1.microsoft.com/is/content/microsoftcorp/microsoft/msc/documents/presentations/CSR/Microsoft-2024-Environmental-Sustainability-Report.pdf.

[14] Jakob Nibler and Thomas Ropars. 2025. Carbon Footprint of Storage in Data Centers: the Impact of Using SSDs for Key-Value Stores. In *2025 IEEE 25th International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. 305–314.

[15] Nvidia. 2024. NVIDIA H100 GPU Datasheet — resources.nvidia.com. https://resources.nvidia.com/en-us-hopper-architecture/nvidia-tensor-core-gpu-datasheet?ncid=no-ncid.

[16] Nvidia. 2025. Product Carbon Footprint (PCF) Summary for NVIDIA HGX H100. https://images.nvidia.com/aem-dam/Solutions/documents/HGX-H100-PCF-Summary.pdf.

[17] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132.

[18] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently Scaling Transformer Inference. *Proceedings of Machine Learning and Systems* (2023).

[19] Sourjya Roy, Shrihari Sridharan, Surya Selvam, and Anand Raghunathan. 2025. KV-CAR: KV Cache Compression using Autoencoders and KV Reuse in Large Language Models. arXiv:2512.06727

[20] Steffen Schlömer, Thomas Bruckner, Lew Fulton, Edgar Hertwich, Alan McKinnon, Daniel Perczyk, Joyashree Roy, Roberto Schaeffer, Ralph Sims, Pete Smith, et al. 2014. Annex III: Technology-specific cost and performance parameters. In *Climate change 2014: Mitigation of climate change: Contribution of working group III to the fifth assessment report of the Intergovernmental Panel on Climate Change*. Cambridge University Press, 1329–1356.

[21] Seagate Technology. 2020. Nytro 3331 Sustainability Report. https://www.seagate.com/files/www-content/global-citizenship/_shared/product-sustainability/nytro-3331-sustainability-report/_shared/pdf/nytro-3331-sustainability-report-en-ca.pdf Product Sustainability Report.

[22] Kun-Woo Shin, Jay H. Park, Moonwook Oh, Yohan Jo, Jaeyoung Do, and Sang-Won Lee. 2025. MatKV: Trading Compute for Flash Storage in LLM Inference. arXiv:2512.22195

[23] Swamit Tannu and Prashant J Nair. 2023. The Dirty Secret of SSDs: Embodied Carbon. *SIGENERGY Energy Informatics Review* 3 (2023).

[24] Yuyang Tian, Desen Sun, Yi Ding, and Sihang Liu. 2026. Cache Your Prompt When It's Green: Carbon-Aware Caching for Large Language Model Serving. arXiv:2505.23970

[25] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[26] Jaylen Wang, Daniel S. Berger, Fiodar Kazhamiaka, Celine Irvene, Chaojie Zhang, Esha Choukse, Kali Frost, Rodrigo Fonseca, Brijesh Warrier, Chetan Bansal, Jonathan Stern, Ricardo Bianchini, and Akshitha Sriraman. 2024. Designing Cloud Servers for Lower Carbon. In *International Symposium on Computer Architecture*.

[27] Carole-Jean Wu, Ramya Raghavendra, Udit Gupta, Bilge Acun, Newsha Ardalani, Kiwan Maeng, Gloria Chang, Fiona Aga, Jinshi Huang, Charles Bai, Michael Gschwind, Anurag Gupta, Myle Ott, Anastasia Melnikov, Salvatore Candido, David Brooks, Geeta Chauhan, Benjamin Lee, Hsien-Hsin Lee, Bugra Akyildiz, Maximilian Balandat, Joe Spisak, Ravi Jain, Mike Rabbat, and Kim Hazelwood. 2022. Sustainable AI: Environmental Implications, Challenges and Opportunities. *Proceedings of Machine Learning and Systems* (2022).

[28] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *Advances in Neural Information Processing Systems* (2023).

[29] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. *Advances in Neural Information Processing Systems* (2024).